

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Departamento de Ciencias e Ingeniería de la Computación

Fundamentos de Programación

Practica 10 Archivos de acceso secuencial

Emplee apuntadores, memoria dinámica, paso de argumentos por referencia, aritmética de apuntadores y archivos textuales (que se pueden leer), no olvide crear sus diagramas de flujo para mostrar su práctica al maestro antes del código y de ejecutarlo. Para ver el funcionamiento, se recomienda almacenar toda la información en su archivo.

1. Escriba un programa que lea una cuartilla tamaño carta de texto desde un archivo y genere una lista de todas las letras que aparecen en el texto junto con el número de veces que cada letra ocurre en la línea. Termina la línea con un punto que sirva como valor centinela. Las letras deben enumerarse en el siguiente orden: la letra más frecuente, la siguiente letra más frecuente, y así sucesivamente. Usar dos matrices (dinámicas), una para contener los números enteros y otra para contener las letras, puede asumir que la entrada utiliza todas las letras minúsculas. Por ejemplo, la entrada (este ejemplo no cumple lo del archivo de entrada, solo es demostrativo)

do be do bo

debe producir una salida similar a la siguiente (que deberá ir al flujo del archivo de salida):

```
Lectura del archivo:  lenguajej.txt
Letra  Numero de ocurrencias
o      3
d      2
b      2
e      1
escritura de resultados:      resultados.txt
```

Su programa necesitará ordenar los arreglos de acuerdo a los valores en una matriz de enteros (un vector de apuntador asignado con memoria dinámica). Esto requerirá que modifique el tipo de función. No puede usar ordenar para resolver este problema sin cambiar la función. Los nombres de archivo deberán terminar con .txt. Salve los resultados en un archivo de texto

2. Empleando estructuras anidadas como las siguientes

```
#define N 50
#define LARGO 40
struct Fecha {
    int dia;
    int mes;
    int anio;
};
```

```

struct Identificacion {
    char nombre[LARGO];
    char apellido[LARGO];
    struct Fecha fechaNacimiento;
    char direccion[LARGO];
    /* lo que haga falta de la direccion */
    char ciudad[LARGO];
};

struct Obrero{
    struct Identificacion datos;
    long sueldobase;
};

```

Haga lo propio para procesar la información de los mismos obreros de su practica 9 pero ahora es necesario almacenar la información de todos sus registros en un archivo de texto. Describa en su desarrollo como se almacena la información en su archivo. NOTA: Cambie los arreglos por apuntadores y emplee memoria dinámica.

3. Cree un programa escrito en lenguaje C que se encarga de manejar los datos de una empresa leídos desde un archivo que los almacena, esta puede tener hasta un máximo de 100 empleados. Para lo anterior se utilizan las siguientes estructuras:

```

struct Obrero {
    int codigo;
    char apellido[30];
    char nombre[25];
    int edad;
    char direccion[35];
    int codigoPostal;
};

struct Departamento {
    int codigoDepartamento;
    char nombredepartamento[35];
};

struct Sueldo {
    int codigo;
    long sueldoBase;
    long otros;
};

struct Auxiliar {
    int codigo;
    char apellido[30];
};

```

Al ejecutarse el programa, esta carga los datos en las siguientes variables:

```

/*arreglo que contiene los datos de cada obrero */
struct Obrero obreros[100];
/*arreglo que contiene el codigo y nombre de cada depto. */
struct Departamento departamentos[30];

```

```

/*arreglo que contiene los sueldos de los empleados */
struct Sueldo sueldos[100];
/* arreglo de enteros */
struct Auxiliar obrerosOrdinarios[100];

```

Ninguna de las variables anteriores debe ser globales, cámbielas por apuntadores y emplee memoria dinámica. Cada obrero al ser ingresado al sistema se le asigna un número (código), este es en orden correlativo. Dicho código se utiliza para identificar al obrero, por ejemplo, en el arreglo sueldos, se ocupa para indicar el sueldo de un obrero cuyo código está indicado. Recuerde emplear un menú para realizar todas las funciones necesarias como ingresar obrero, cambiarlo, eliminarlo, buscarlo, etcétera, así como cargar la información desde su archivo de texto y almacenarla también, para finalizar cree una función que haga un respaldo de su archivo de base de obreros.

Ejemplo:

Arreglo obreros:

codigo	apellido	nombre	edad	direccion	codigodepto
1	Alexander	Nicolai	25	O'Higgins 54	3
2	Sanchez	Alfredo	32	R. Correa 1050	4
3	Valencia	Josefina	29	I. C. Pinto 33	3
4	Gonzalez	Ramiro	25	Mardones 166	2
5	Gonzalez	Miguelina	31	Bories 1420	3
6			..		..
:			..		..
:			..		..
:			..		..
-1			..		..
-1			..		..
-1			..		..

Arreglo departamentos:

coddepto	nomdepto
1	Administración
2	Mantención
3	Informática
.	
.	
.	
-1	
-1	

Arreglo sueldos:

codigo	sueldobase	otros
1	230.000	51.000
2	180.000	33.000
3	200.000	40.000
4	140.000	23.000
.		

.		
.		
-1		...
-1		...

Entonces Josefina Valencia del departamento de Informática gana 200.000 + 40.000 pesos.
(los datos con -1 indican vacíos o nulos).